Transport Layer:

4.1 Transport Services: Services provided to Upper layer, Transport service primitives, Berkeley Sockets Elements of

4.2 Transport protocols: Addressing, Error and Flow Control, Multiplexing, Crash Recovery,
4.3 Connection management. Internet Transport Protocols: TCP: Service Model, TCP Protocol, Segment header,
4.4 TCP Connection management,
4.5 TCP Sliding Window, Timer management, TCP Congestion Control. UDP: Protocol, UDP Header

The transport service

TCP service is obtained by both the sender and the receiver creating end points, called sockets. Each socket has a socket number (address) consisting of the IP address of the host and a 16-bit number local to that host, called a port. A port is the TCP name for a TSAP.

Addressing

When an application (e.g., a user) process wishes to set up a connection to a remote application process, it must specify which one to connect to. In the Internet, these endpoints are called ports. TSAP (Transport Service Access Point) to mean a specific endpoint in the transport layer. End points in the network layer are called NSAPs (Network Service Access Points).

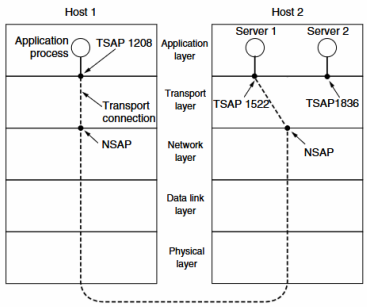


Figure 6-8. TSAPs, NSAPs, and transport connections.

A port number is a 16-bit logical address used at the transport layer to identify a specific application or service running on a computer. Port numbers are used by TCP and UDP protocols to deliver data to the correct application. The port number ranges from **0 to 65535** (2^1 possible values).

Types of Port Numbers:

1. **Well-Known Ports (0–1023):** These ports are assigned to commonly used services and protocols. These port numbers are reserved for standard services that can usually only be started by privileged users. The list of well-known ports is given at www.iana.org. Over 700 have been assigned. A few of the better-known ones are listed.

Port	Protocol	Use
20, 21	FTP	File transfer
22	SSH	Remote login, replacement for Telnet
25	SMTP	Email
80	HTTP	World Wide Web
110	POP-3	Remote email access
143	IMAP	Remote email access
443	HTTPS	Secure Web (HTTP over SSL/TLS)
543	RTSP	Media player control
631	IPP	Printer sharing

2. **Registered Ports (1024–49151):** These ports are registered for specific applications and services.
    - Port 3306 – MySQL
    - Port 5432 – PostgreSQL
    - Port 8080 – Commonly used for alternative HTTP services
3. **Dynamic or Private Ports (49152–65535):** These ports are generally used temporarily by client applications for communication with servers. They are also called ephemeral ports.

TCP Connection establishment and Release

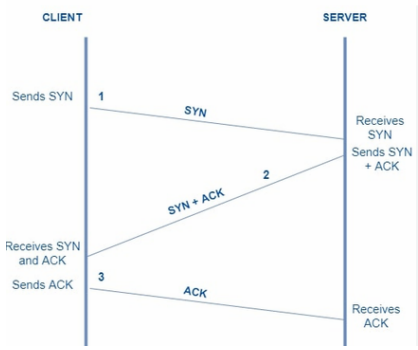
TCP (Transmission Control Protocol) is a connection-oriented transport layer protocol. It establishes a connection before data transmission and releases the connection after communication is completed.

1. TCP Connection Establishment (Three-Way Handshake)

TCP transmits data in full-duplex mode. When two TCPs in two machines are connected, they are able to send segments to each other simultaneously. This implies that each party must initialize communication and get approval from the other party before any data are transferred.
The connection establishment in TCP is called three-way handshaking.

TCP establishes a connection using three steps:

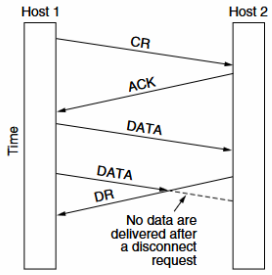
1. **SYN:** The client sends the first segment, a SYN segment, in which only the SYN flag is set. This segment is for synchronization of sequence numbers. It consumes one sequence number. When the data transfer starts, the sequence number is incremented by 1. SYN segment carries no real data.
2. **SYN-ACK:** The server sends the second segment, a SYN + ACK segment, with 2 flag bits set:SYN and ACK. This segment has a dual purpose. It is a SYN segment for communication in the other direction and serves as the acknowledgment for the SYN segment.
3. **ACK:** The client sends the third segment. This is just an ACK segment. It acknowledges the receipt of the second segment with the ACK flag and acknowledgment number field. Note that the sequence number in this segment is the same as the one in the SYN segment.



Connection Termination

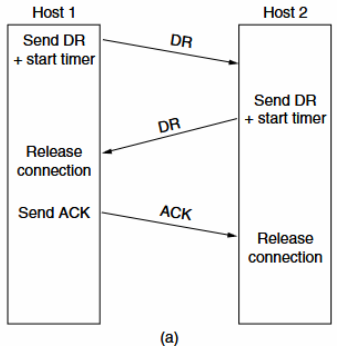
There are two styles of terminating a connection: asymmetric release and symmetric release

1. Asymmetric release: Asymmetric release is the way the telephone system works: when one party hangs up, the connection is broken. Asymmetric release is abrupt and may result in data loss. After the connection is established, host 1 sends a segment that arrives properly at host 2. Then host 1 sends another segment. Unfortunately, host 2 issues a DISCONNECT before the second segment arrives. The result is that the connection is released and data are lost.



ure 6-12. Abrupt disconnection with loss of data.

2. Symmetric release means that each side of the connection must independently agree to close its connection. The connection is released only after both hosts have finished sending their data and have agreed to disconnect. Unlike asymmetric release, symmetric release allows data transmission in one direction to continue even after the other side has finished sending data



Error and Flow Control

After connection is establishment, connections management during data transmission involves the two main issues : error control and flow control.

- Error control: Ensures that data reaches the receiver reliably, with no loss or corruption, as required.
- Flow control: Prevents a fast sender from transmitting data faster than a slow receiver can process it.

TCP connections: Often require large sliding windows because the bandwidth-delay product can be much larger than one segment. Stop-and-wait would severely reduce throughput.

Buffers temporarily store data during transmission.

- Sender buffers: Store transmitted but unacknowledged segments so they can be retransmitted if lost.
- Receiver buffers: Store incoming segments until they can be processed. If no buffer is available, a segment may be discarded and retransmitted later.

Buffer allocation strategies

Strategy	Description	Advantage / limitation
Fixed-size buffers	One buffer per segment	Simple when segment sizes are similar
Variable-size buffers	Buffers match different segment sizes	Better memory utilization but more complex management
Circular buffer per connection	One large circular buffer for each connection	Simple organization, but uses memory efficiently mainly under heavy load

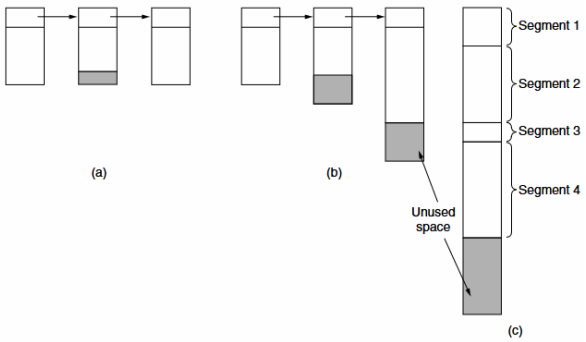


Figure 6-15. (a) Chained fixed-size buffers. (b) Chained variable-sized buffers. (c) One large circular buffer per connection.

Multiplexing

Multiplexing is the process of sharing one network connection among multiple transport connections. It is used in the transport layer to allow multiple application processes to communicate using the same network address.

1. Multiplexing

- Multiple transport connections share a single network connection or IP address.
- When a segment arrives, the transport layer uses port numbers to identify the correct application process.
- Example: Multiple applications on your computer use the same IP address to communicate with different servers.

2. Inverse Multiplexing

- A single transport connection distributes its traffic across multiple network paths.
- Traffic may be distributed in a round-robin manner to increase bandwidth or improve reliability.
- With \(k\) network connections, bandwidth may increase by a factor approaching \(k\), depending on the network conditions and implementation.
- Example: SCTP can use multiple network interfaces in a multi homed connection. Similar link-layer techniques combine multiple low-speed links into one higher-capacity link.

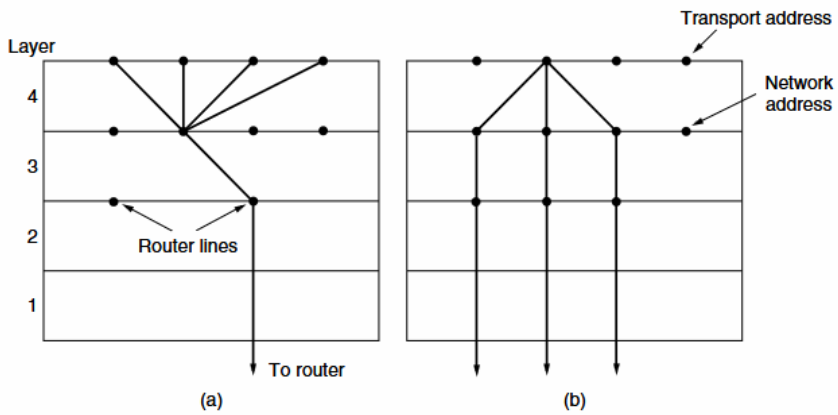


Figure 6-17. (a) Multiplexing. (b) Inverse multiplexing.

Crash Recovery

Crash recovery is the process of restoring communication after a host or router crashes during data transmission.

1. Recovery from Network and Router Crashes

Recovery is relatively straightforward because transport protocols such as TCP can detect lost segments and retransmit them.

2. Recovery from Host Crashes

Host crash recovery is difficult because the server may lose its connection state after restarting.

Example: A client transfers a large file to a server using the stop-and-wait protocol. If the server crashes during transmission, it may lose information about the segments received and acknowledged.

The client can be in two states:

- **S0:** No segment is awaiting acknowledgement.
- **S1:** One segment has been sent but not yet acknowledged.

3. Events During Crash Recovery

Three events can occur at the server:

- **A (Acknowledgement):** The server acknowledges the received segment.
- **W (Write):** The server delivers the segment to the application process.
- **C (Crash):** The server crashes and loses its transport state.

The six possible event sequences are:

Sequence	Meaning
AC(W)	Acknowledgement, crash, and no subsequent write
AWC	Acknowledgement, write, then crash
C(AW)	Crash before acknowledgement or write
C(WA)	Crash before write or acknowledgement
WAC	Write, acknowledgement, then crash
WC(A)	Write, crash before acknowledgement

Parentheses indicate that the enclosed event cannot occur after the crash.

4. Problems in Crash Recovery

Consider a client strategy that always retransmits the last segment after learning of a crash.

For the sequence **AWC**:

1. The server acknowledges the segment.
2. The server writes the segment to the application.
3. The server crashes.
4. The client retransmits the segment after recovery.
5. The application may receive the same data twice.

Result: An undetected duplicate segment.

If the server acknowledges first and crashes before writing the segment, data may instead be lost because the client believes the segment was delivered.

Strategy used by sending host	Strategy used by receiving host: First ACK, then write			Strategy used by receiving host: First write, then ACK		
	AC(W)	AWC	C(AW)	C(WA)	W AC	WC(A)
Always retransmit	OK	DUP	OK	OK	DUP	DUP
Never retransmit	LOST	OK	LOST	LOST	OK	OK
Retransmit in S0	OK	DUP	LOST	LOST	DUP	OK
Retransmit in S1	LOST	OK	OK	OK	OK	DUP

OK = Protocol functions correctly
DUP = Protocol generates a duplicate message
LOST = Protocol loses a message

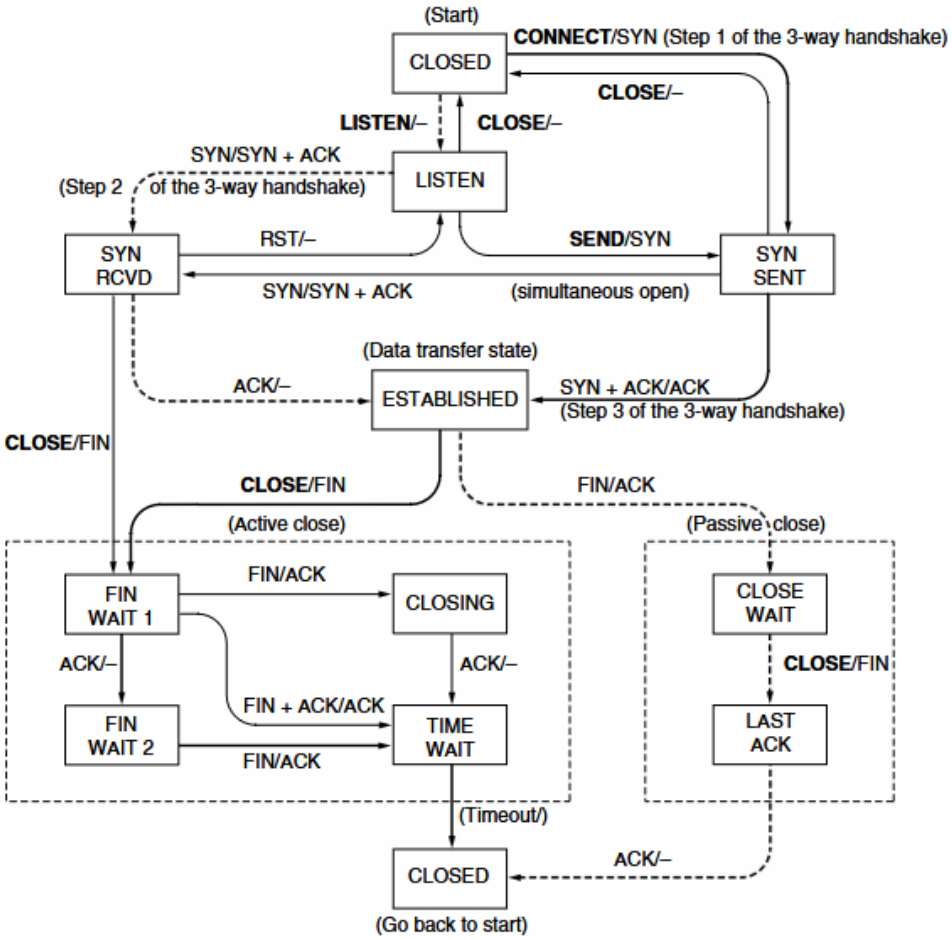
TCP Connection Management Model:

The steps required to establish and release connections can be represented in a finite state machine with the 11 states listed in Fig.

State	Description
CLOSED	No connection is active or pending
LISTEN	The server is waiting for an incoming call
SYN RCVD	A connection request has arrived; wait for ACK
SYN SENT	The application has started to open a connection
ESTABLISHED	The normal data transfer state
FIN WAIT 1	The application has said it is finished
FIN WAIT 2	The other side has agreed to release
TIME WAIT	Wait for all packets to die off
CLOSING	Both sides have tried to close simultaneously
CLOSE WAIT	The other side has initiated a release
LAST ACK	Wait for all packets to die off

In each state, certain events are legal. When a legal event happens, some action may be taken. If some other event happens, an error is reported.Each connection starts in the CLOSED state. It leaves that state when it does either a passive open (LISTEN) or an active open (CONNECT). If the other side does the opposite one, a connection is established and the state becomes ESTABLISHED. Connection release can be initiated by either side. When it is complete, the state returns to CLOSED. The finite state machine itself is shown in Fig

Each line in Fig. 6-39 is marked by an event/action pair. The event can either be a user-initiated system call (CONNECT, LISTEN, SEND, or CLOSE), a segment arrival (SYN, FIN, ACK, or RST), or, in one case, a timeout of twice the maximum packet lifetime. The action is the sending of a control segment (SYN, FIN, or RST) or nothing


(Start)
CLOSED
CONNECT/SYN (Step 1 of the 3-way handshake)
CLOSE/–
LISTEN/–
CLOSE/–
SYN/SYN + ACK
(Step 2 of the 3-way handshake)
LISTEN
SYN RCVD
RST/–
SEND/SYN
SYN SENT
SYN/SYN + ACK
(simultaneous open)
(Data transfer state)
ACK/–
ESTABLISHED
SYN + ACK/ACK
(Step 3 of the 3-way handshake)
CLOSE/FIN
CLOSE/FIN
FIN/ACK
(Active close)
(Passive close)
FIN WAIT 1
FIN/ACK
CLOSING
CLOSE WAIT
ACK/–
ACK/–
CLOSE/FIN
FIN + ACK/ACK
FIN WAIT 2
TIME WAIT
LAST ACK
FIN/ACK
(Timeout/)
ACK/–
CLOSED
(Go back to start)